01 / THE IDEA

From card to resolution.

An AI payment-support teammate that owns a complaint from purchase identification to investigation, permitted action, and verified follow-up.

“Scan your card. Explain the problem. Our AI investigates, acts, and follows through.”

The problem

“Money was deducted, but my order failed.” Resolving this requires someone to find the purchase, compare payment and order records, check refund policy, take action, and follow up. A useful answer alone does not finish the job.

The solution

Scan-to-Resolve coordinates those steps as one persistent customer-service case. Sarvam understands the complaint and chooses tools. Backend rules control refunds. Humans handle exceptions with the evidence already gathered.

Customer entry point	Business outcome
Scan the last four digits and choose a purchase	Reduce typing and identify the relevant merchant order.
Explain the issue naturally	Investigate payment, fulfillment, refund history, and policy.
Follow a live case timeline	Track the action and outcome without repeating the complaint.

Scope and positioning
Built for an integrated merchant using Paytm. The scan filters account-linked purchases; it does not retrieve arbitrary bank transactions or prove card ownership.

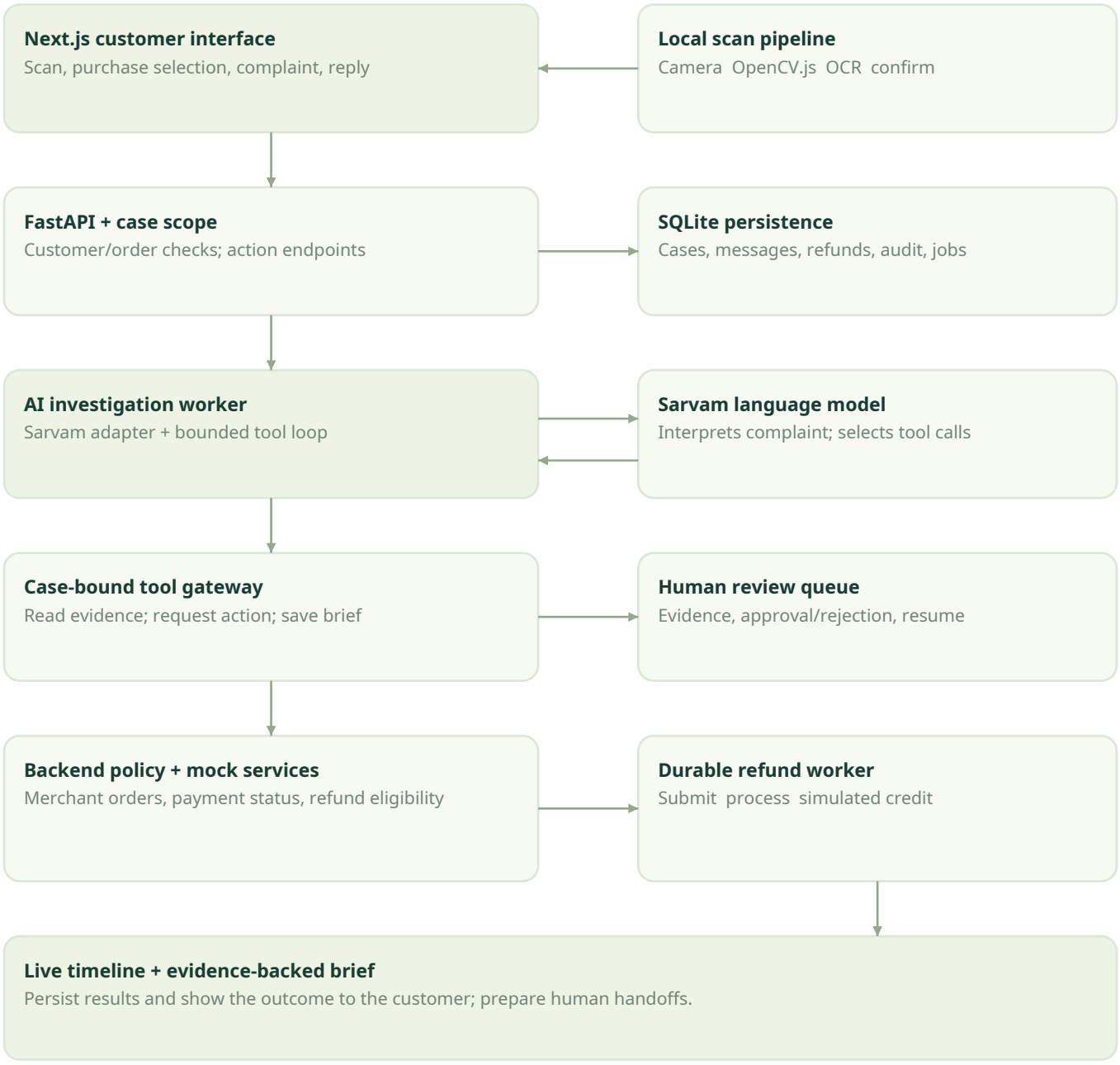
Why this fits the challenge

Understand context choose tools execute permitted actions track outcomes collaborate with humans. The memorable scan starts the experience; persistent workflow ownership delivers the value.

02 / HOW THE PIECES CONNECT

System architecture

The model selects tools. The application binds them to the selected case, validates every action, and records the results.



Real AI, simulated payments
Sarvam inference, tool execution, policy checks, persistence, and the interface are real. Paytm responses, merchant data, and bank-credit events are simulated. No real funds move.

03 / END-TO-END EXPERIENCE

The customer journey

1. Identify the purchase

Show only the last four digits. OpenCV improves the camera frame and finds a likely card region. Browser-local OCR reads the visible digits. The customer confirms the result; manual entry is available.

2. Select the affected order

The interface filters purchases already linked to the demo customer. The customer chooses the merchant order, establishing the scope for every subsequent tool call.

3. Explain the issue naturally

Sarvam receives redacted complaint text and case-bound tool results. English and Hinglish scenarios have been tested live. If the intent is unclear, the teammate can ask a question and wait for a reply.

4. Investigate before acting

The teammate gathers fulfillment, payment, existing-refund, and policy evidence. Every executed tool result is saved with an inspectable evidence reference.

5. Take the permitted next step

Request an eligible mock refund, track an existing refund, answer without issuing a refund, ask for clarification, or escalate. A successful payment with a cancelled order differs from a failed or pending payment.

6. Follow through

A persistent worker tracks the simulated refund through processing and credit confirmation. Refreshing the browser or restarting the backend does not erase the case or its scheduled work.

Privacy boundary
Card photos stay in the browser. The model context excludes card digits and unrelated orders. Common full-card-number and labelled PIN/CVV/OTP patterns are redacted; customers should still avoid entering sensitive details.

04 / WHAT IS BUILT TODAY

The implementation stack

Layer	Technology	Responsibility
Customer interface	Next.js + React + TypeScript	Scan, purchase selection, complaint, clarification, tracking.
Computer vision	OpenCV.js	Grayscale, blur, edge detection, likely card-region crop.
Local recognition	Tesseract OCR	Read four visible digits locally; customer confirms.
Application API	FastAPI / Python	Cases, scoped tools, review decisions, configuration.
AI provider	Sarvam 105B	Understand intent and select function calls.
Provider adapter	Chat Completions v2	Send tools, execute locally, replay tool results.
Policy engine	Backend business rules	Eligibility, amount limit, execution-time rechecks.
Background execution	Python workers	AI investigation, due transitions, refund tracking.
Storage	SQLite	Cases, orders, messages, tool evidence, refunds, events.
Human workspace	Next.js review queue	Inspect evidence, approve/reject, resume.
Payment integration	Mock Paytm behavior	Payment lookup, refund acceptance, processing, credit.

Provider and operating boundaries

Sarvam is the selected provider. The OpenAI adapter remains available as an explicit alternative; there is no silent fallback. Each investigation has a bounded tool loop and records token usage.

Current local prototype
Identity is a fixed demo account, and reviewer controls are shared for demonstration. Live AI is paused to preserve credits until the demo. Production authentication and reviewer role separation are not implemented.

05 / CONTROLLED AUTONOMY

Tools, decisions, safeguards

Tool	Purpose
get_order	Read selected-order fulfillment and merchant evidence.
get_payment_status	Read mocked payment status and amount.
get_existing_refunds	Detect existing refunds and prevent duplicates.
evaluate_refund_policy	Return the authoritative backend eligibility decision.
request_refund / track_refund	Request a permitted mock refund or follow an existing one.
ask_customer / escalate_case	Pause for clarification or route the case to a human.
finish_case_brief	Save a summary linked to actual recorded evidence IDs.

Decision rules

Verified situation	System behavior
Payment succeeded + cancelled order + policy permits	Request the eligible refund.
Refund already exists	Track it; do not create a second refund.
Pending/failed payment needing resolution	Reconcile or escalate; no blind refund.
Fulfilled order or conflicting evidence	Human review under the sample policy.
Refund exceeds ₹1,000 automatic limit	Human approval before execution.
Information-only request / unclear intent	Answer without refund / ask a clarification.

Backend controls: the model cannot supply arbitrary account IDs, merchant IDs, order IDs, refund amounts, or approval flags. Refund requests require evidence reads and fresh eligibility checks. Unique case/refund records prevent duplicate actions.

The ₹1,000 limit is a sample merchant policy, not a Paytm rule.

06 / WORKFLOW + EVIDENCE

Ownership and human teamwork

Persistent case lifecycle

AI investigation
Case opened AI queued AI investigation tool-selected next step. Exceptions pause for customer clarification, AI retry, or human review. Information-only cases can finish without a financial action.

Refund lifecycle
Eligible request submission scheduled mock refund accepted processing simulated bank credit resolved. Backend policy is rechecked before execution; refund tracking continues if the model's final summary fails.

A handoff that is ready for a decision

Issue: Customer requests a ₹1,499 refund.
Evidence: Successful payment, cancelled order, no existing refund.
Reason: Amount exceeds the sample automatic limit.
Decision: Approve refund or close without a refund.

A human inspects the record and decides. Approval resumes the existing workflow after eligibility checks. The customer does not need to repeat the complaint or open another case.

Standout feature: evidence-backed case brief

- **Intent and findings:** a customer-facing summary of what the AI understood and checked.
- **Source references:** click an evidence number to inspect the executed tool result.
- **Validated citations:** fabricated evidence IDs are rejected by the backend.
- **Portable handoff:** copy or download the brief and its source records for review.
- **Clear time context:** recommendations are timestamped; the live state and timeline show later changes.

Judges can inspect what the teammate checked, what action it requested, and what outcome the system confirmed. This is observable workflow evidence, not private model reasoning.

07 / MEASURABLE OUTCOMES

Proof, impact, and next steps

Verified prototype evidence

• Live Sarvam: a Hinglish refund request selected investigation tools and completed the simulated refund lifecycle.

• Live Sarvam: an explicit status-only request finished without creating a refund.

• Browser verification: a saved pending-payment case produced an answer with four inspectable evidence sources and no refund.

• **29 automated tests pass**, covering provider loops, duplicate protection, blocked refunds, clarification, provider errors, and evidence validation. The production frontend build passes.

These checks demonstrate working functionality. They are not a broad accuracy benchmark or evidence of real bank settlement.

Measure these outcomes in the pitch

Metric	How to measure it
Correct-action rate	Compare each case's actual action with a predefined expected outcome.
Autonomous resolution rate	Eligible cases completed without human intervention / eligible cases.
Duplicate refunds	Count repeated actions for the same intended operation.
Time to outcome	Report AI investigation time separately from simulated refund time.
Handoff completeness	Check for evidence, escalation reason, and a clear decision request.
Model usage per case	Use recorded tokens; confirm actual spend in the provider dashboard.

Path from prototype to production

Connect authorized merchant Paytm APIs and the real order system. Add customer authentication and reviewer roles, signed webhooks, timeout reconciliation, external-call idempotency, a production database/job queue, and representative evaluations.

Preserve the distinction between refund acceptance, provider processing, and customer bank credit. Only claim each stage when the relevant system confirms it.

08 / PRESENTATION PLAYBOOK

The pitch and demo

Three-minute demonstration

Time	Show the outcome
0:00-0:25	Explain the payment-support problem and the teammate's ownership.
0:25-1:30	Scan select order Hinglish complaint tools mock refund tracking.
1:30-2:00	Show "check status, do not refund" producing an answer without a refund.
2:00-2:30	Show approval-required handoff, inspect evidence, approve, and resume.
2:30-3:00	Open the evidence brief; present measured results and integration roadmap.

Ready-to-deliver pitch

When a customer says, "Money was deducted, but my order failed," resolving the issue requires more than a helpful reply. Someone must check the payment, inspect the order, apply policy, take action, and follow up.

We built **Scan-to-Resolve**, an AI payment-support teammate. The customer scans the last four digits of their card, selects their purchase, and explains the problem naturally - in English, Hindi, or Hinglish.

Sarvam chooses the investigation tools. Our backend validates the evidence and controls every refund action. Eligible cases move forward automatically; exceptions reach a human with the evidence already gathered.

The teammate continues tracking the case after its first response. Every recommendation has inspectable sources, and human approvals resume the same workflow.

Today, real Sarvam inference drives the investigation while Paytm and bank outcomes are simulated. Our prototype demonstrates automatic resolution, intent-sensitive action, duplicate protection, and human collaboration.

Scan-to-Resolve turns a payment complaint into a tracked, evidence-backed workflow - with a clear next step and an owner until completion.

Integration references: Sarvam Chat Completions v2 · Paytm refund lifecycle